

AXIOM MANIFESTO

A Conceptual Framework for Agent-Native Commerce
on Kaspia + Igra

Version 1.0 | June 2026

Independent Research Project by Andreas (pay.igra)

"The Internet was built for humans.
The next economy will be built for agents."

Axiom Manifesto

A Conceptual Framework for Agent-Native Commerce on Kaspas + Igra

Independent research. Not an official roadmap, protocol specification or endorsement by Kaspas, Igra, Argent, Kaskad or any ecosystem entity.

1. Introduction

The Internet Was Built For Humans

The modern internet was designed around human cognition. Humans authenticate themselves. Humans read information. Humans authorize payments. Humans settle disputes.

The next wave of economic activity will increasingly be delegated to autonomous software agents.

These agents will not merely retrieve information. They will negotiate, coordinate, purchase, lend, borrow and settle value on behalf of individuals and organizations.

This shift requires more than faster blockchains.

It requires a coherent architecture for trust, authorization, truth, execution and settlement.

Axiom proposes a conceptual framework for discussing this emerging agent economy.

The Limitation

Current stacks rely on patching account-based chains (ERC-4337/7702), forcing agents into a congested, shared global state.

The Axiom Shift

We introduce a **covenant-native composable state** model, treating agents as actors within a decomposable state machine. This directly extends the technical vision pioneered by Kaskad — where composable state transitions enable parallel, non-interfering economic flows — into a complete semantic framework for agent-native commerce.

Where Kaskad provides the machinery for covenant-native composable state, Axiom provides the **economic vocabulary** that makes that machinery operable for autonomous agents.

The Outcome

By leveraging UTXO locality and atomic transitions, we enable high-frequency economic flows that are structurally hardcoded, not added as an afterthought.

2. Why Layers Matter

The history of computing shows that complex systems emerge through abstraction layers.

The internet evolved through:

- TCP/IP (transport)
- DNS (naming)
- HTTP (application protocol)
- Applications (user-facing)

Financial systems evolved through:

- Identity (who)
- Accounts (where)
- Payment networks (how)
- Settlement systems (when final)

Agent-native commerce may require similar abstraction layers.

Axiom organizes these capabilities into five conceptual layers.

3. The Five Layers

Layer 1 — Orchestration

Purpose: Agents need a way to express objectives and coordinate execution.

Primitives:

- `intent.igra` — Express economic objective
- `agents.igra` — Agent registry and discovery

- `mcp.igra` — Model Context Protocol for inter-agent communication

Technical Context:

Recent work such as Argent explores how complex workflows may be decomposed into multiple covenant transitions rather than executed inside a single monolithic contract. This suggests that future applications may increasingly resemble orchestrated flows of state transitions.

Kaskad's approach to **covenant-native composable state** demonstrates that parallel, non-interfering transitions are not merely possible — they are the native execution model for agent-native systems on Kaspera.

Intent becomes the starting point. Orchestration becomes the routing layer.

Layer 2 — Identity & Governance

Purpose: Agents must act on behalf of principals. Without verifiable authorization there is no accountable commerce.

Primitives:

- `auth.igra` — Scoped, revocable permissions
- `identity.igra` — Verifiable agent identity
- `governance.igra` — Mutable policy parameters

Technical Context:

Every economic action requires permission. Human owners need revocable authorization, scoped permissions and accountability.

Identity transforms autonomous behavior into delegated action.

Layer 3 — Data & Truth

Purpose: An agent is only as reliable as the information it acts upon.

Primitives:

- `data.igra` — Structured state schemas
- `oracle.igra` — External truth attestation
- `logs.igra` — Immutable audit trail

Technical Context:

Future agent systems require trusted data, signed observations and auditable decision trails.

The value of logs increases as autonomous systems become accountable for financial outcomes. Auditability becomes a first-class requirement rather than an afterthought.

Layer 4 — Economic Execution

Purpose: Agents require economic action. Not merely computation. Action.

Primitives:

- `pay.igra` — Value transfer
- `escrow.igra` — Conditional fund locking
- `settle.igra` — Final confirmation
- `supply.igra` — Liquidity provision
- `borrow.igra` — Collateralized debt
- `repay.igra` — Debt settlement

Technical Context:

Economic execution represents the transition from decision to value transfer.

Escrow introduces conditional commitment. Settlement introduces finality. Lending primitives enable persistent economic relationships between agents.

Layer 5 — Infrastructure

Purpose: Higher layers depend on shared infrastructure.

Primitives:

- `kskd.igra` — Native Kasper denomination and fee handling
- `custody.igra` — Multi-sig and treasury management
- `kurrent.igra` — Real-time exchange rate and cross-chain settlement

Technical Context:

Infrastructure provides state synchronization, custody models and information transport.

These components are largely invisible to users but critical to system reliability.

4. The Agentic Pay Example

A simple workflow illustrates the interaction between layers.

User objective: Purchase a laptop below \$1,200.

Flow:

```
intent → auth → data → escrow → pay → settle → logs
```

The agent:

1. Receives an objective
2. Verifies authorization
3. Retrieves trusted information (price, availability)
4. Reserves funds in escrow
5. Executes payment upon delivery confirmation
6. Finalizes settlement
7. Records the audit trail

The flow demonstrates how multiple primitives combine into a coherent economic action.

5. Axiom and Decomposable Computation

Recent research in the Kaspas ecosystem explores multi-contract architectures and decomposable computation.

Rather than executing all logic inside a single contract, applications may increasingly be expressed as sequences of covenant transitions — each verifying the next, each carrying its own state. Kaskad's work on **covenant-native composable state** provides the technical foundation for this paradigm.

Axiom does not attempt to specify these transitions.

Instead, it proposes a vocabulary for discussing the roles these transitions play in agent-native commerce. Where Kaskad builds the engine, Axiom builds the dashboard — the semantic layer that makes composable state economically meaningful.

The framework focuses on semantic structure rather than contract implementation.

6. Conclusion

The purpose of Axiom is not to predict the future roadmap of Kaspa or Igra.

Its purpose is to provide a common language for discussing agent-native commerce before the infrastructure fully exists.

Technology evolves. Shared vocabulary often emerges slowly.

Axiom is an attempt to map that vocabulary before the market demands it.

Appendix: The 17 Primitives

#	Primitive	Layer	Function
1	intent.igra	L1	Express economic goal
2	agents.igra	L1	Agent registry & discovery
3	mcp.igra	L1	Inter-agent communication
4	auth.igra	L2	Scoped permissions
5	identity.igra	L2	Verifiable agent ID
6	governance.igra	L2	Mutable policy
7	data.igra	L3	Structured state
8	oracle.igra	L3	External truth
9	logs.igra	L3	Audit trail
10	pay.igra	L4	Value transfer
11	escrow.igra	L4	Conditional locking
12	settle.igra	L4	Final confirmation
13	supply.igra	L4	Liquidity provision
14	borrow.igra	L4	Collateralized debt
15	repay.igra	L4	Debt settlement
16	kskd.igra	L5	Native fee handling

17	<code>custody.igra</code>	L5	Treasury management
18	<code>kurrent.igra</code>	L5	Real-time exchange

Built on Axioms. Designed for Autonomous Commerce.

The Internet was built for humans. The next economy will be built for agents.